

Programming Assignment – Part I

MENG544/ROBO554 Robot Mechanics: Kinematics, Dynamics, and Control

Dr. George Kontoudis

Department of Mechanical Engineering

Colorado School of Mines

Fall 2026

Overview and Expectations

Introduction

The purpose of this programming assignment is to create a core set of MATLAB functions and models that implement the main topics covered in the course. Although the programming assignment will span the full semester, it will be split into separately graded deliverables. Each deliverable depends on the successful completion of the prior deliverables, so it is important that you correct any errors that may occur. This is a daunting task that will require significant effort throughout the semester.

Plagiarism and Code Copying

Copying code is strictly forbidden. You may and should work together to specify the overall algorithm to be coded or to help with debugging, but the coding itself must be your own. All submissions must reflect the individual's effort to develop them. The learning from this assignment is by doing, debugging, and making it work. Copying does not help you learn. Thus, direct copying of functions, reports, and the like will be penalized strictly per university policy. The first offense results in an F for the entire course, the second offense results in an F for the course and a 1 year suspension, while the third offense results in expulsion.

Coding Style

This programming assignment will consist of writing MATLAB functions that are intended to be useful in the remaining of this course, future courses, and research. Thus, the style of these functions is critical for future usability. To enforce good style practices, part of the grade assigned for each function will be based on following good coding style and readability. Not only does this facilitate easy grading and future usability, but also helps with debugging and reduces the number of coding errors made.

For style guidelines see: <https://www.mathworks.com/matlabcentral/fileexchange/46056-matlab-style-guidelines-2-0> (enter your Mines email at Mathworks webpage and login with your credentials to get access). At a minimum, the script file shall start with a code block that describes what the function does and shall be formatted as in the following example:

```
% myFunctionName: Short description of what this function does.
%
% [output1, output2] = myFunctionName(input1, input2)
% A multiline description of the function would be appropriate.
%
% output1 = description of the first output (include units if appropriate)
% output2 = description of the second output (include units if appropriate)
%
% input1 = description of the first input (include units if appropriate)
% input2 = description of the second input (include units if appropriate)
%
% Your Name
% Your Student Number
% Course Number
% Date
function [output1, output2] = myFunctionName(input1, input2)
% YOUR FUNCTION CODE GOES HERE
end
```

Grading

The programming assignment is worth 20% of your total grade. Each function written will be graded on a three-point scale. One point will be awarded for style and readability, one point for effort, and one point for correctness.

- Upload all functions your code depends on that you write. For example in the second submission you must include your first submission. If your prior submissions had errors, you need to fix them or else the undesired functionality will propagate through the full semester.
- Use the function names I specify for each deliverable, or they will not be graded.

Time Management and Debugging

Together these programming assignments are 20% of the total course grade. As such, they will take time to complete. As the course progresses, you'll gain the information necessary to implement functions sequentially. **It is highly recommended that you work on the programming assignments in parallel with the weekly homework assignments.**

Coding based assignments are both nice and frustrating. Nice in that known test cases provided or generated by you can be used to verify if the implementation is correct. Frustrating in that either it works or it does not. I suggest that for every function you write you verify its correctness against a few known test cases. For example, if one function computes the forward transformation and another is its inverse transformation, verify that the inverse of the function of a value is the original value.

Function Testing

A testing script will be provided. It contains representative test cases for each function. You are expected to use the test script to assist the debugging and verification of your functions prior to submission.

Deliverable 1: Forward Kinematics

The following functions shall be submitted in a single zip folder uploaded to Canvas. A test-script will be provided on Canvas. Final grading will be similar to the testing script, but with slightly different test cases. If you cannot pass the testing script, you will not do well in the grading.

Function Call	Description
<code>R = rotX(theta)</code>	Returns a rotation matrix describing a rotation about the X axis (theta in radians).
<code>R = rotY(theta)</code>	Returns a rotation matrix describing a rotation about the Y axis (theta in radians).
<code>R = rotZ(theta)</code>	Returns a rotation matrix describing a rotation about the Z axis (theta in radians).
<code>R = rpy2Rot(roll, pitch, yaw)</code>	Returns a rotation matrix corresponding to a roll, pitch yaw encoded rotation. Note RPY is defined as the set of orthogonal rotations $\text{rotZ}(\text{yaw})\text{rotY}(\text{pitch})\text{rotX}(\text{roll})$.
<code>[roll, pitch, yaw] = rot2RPY(R)</code>	Returns the roll, pitch and yaw corresponding to a given rotation matrix. It should return the two valid solutions corresponding to the $+\text{sqrt}$ and $-\text{sqrt}$. Each output is then a $[2 \times 1]$ vector with the plus solution on top. <i>Hint:</i> For the degeneracy case assume $\text{yaw} = [0;0]$.
<code>X = cpMap(w)</code>	Returns the matrix packing of the cross product operator, e.g., given vectors w, v ; $w \times v = \text{cross}(w)*v = X*v$.
<code>R = angleAxis2Rot(Omega)</code>	Returns the rotation matrix encoded by an angle-axis ($\text{Omega} = \text{theta}*k$) rotation of theta radians about the unit vector k axis.
<code>[Omega] = rot2AngleAxis(R)</code>	Returns the angle times axis ($\text{theta}*k$) corresponding to a rotation matrix. Take care to handle rotations where $\sin(\text{theta})$ is approximately zero properly. This includes both small angles and pi multiples.
<code>Q = rot2Quat(R)</code>	Returns the quaternion $[\text{q0};\text{q_vec}]$ that corresponds to the rotation matrix.
<code>R = quat2Rot(Q)</code>	Returns the rotation matrix that corresponds to the quaternion, stacked $[\text{q0};\text{q_vec}]$.
<code>Q = multiplyQuat(Q_left, Q_right)</code>	Returns the quaternion product between two quaternions that are stored in 4×1 vectors as $[\text{q0};\text{q_vec}]$.
<code>H = twist2Transform(t)</code>	Returns the homogeneous transformation matrix corresponding to a 6 element twist vector. The twist should be stacked $[\text{v};\text{w th}]$.
<code>t = transform2Twist(H)</code>	Returns the twist vector corresponding to the provided homogeneous transform matrix. The twist should be stacked $[\text{v};\text{w th}]$.
<code>dual_quat= pq2DualQuat(pos, quat)</code>	Returns a dual quaternion stored as a structure with members <i>rot</i> , a 4×1 unit quaternion staked $[\text{q0};\text{q_vec}]$, and <i>disp</i> , a 4×1 quaternion stacked $[\text{q0};\text{q_vec}]$ that encodes the dual rotation-mixed translation.
<code>[pos,quat] = dualQuat2PQ(dual_quat)</code>	Returns a position and a quaternion that represents the transform encoded in the dual quaternion structure, which has members <i>rot</i> and <i>disp</i> .

Function Call	Description
<pre>dual_quat = multiplyDualQuat(dual_quat_left, dual_quat_right)</pre>	Returns the dual quaternion product between two dual quaternions that are stored in the dual quaternion structure, which has members <i>rot</i> and <i>disp</i> , and acts as a chained transformation.
<pre>H = dhTransform(a, d, alpha, theta)</pre>	Returns the homogeneous transform corresponding to the provided DH parameters for a link.